SMU SINGAPORE MANAGEMENT UNIVERSITY
School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

Computational Thinking

- Decomposition
- Pattern Recognition
- Abstraction
- Algorithmic Thinking

This class

- Learn pattern recognition with Python exercises

© MITB, SMU

Pattern Recognition

Something we're all familiar with

- Find the general term of the sequences below:
  - 1, 3, 5, 7, …
  - 1, 3, 6, 10, …
  - 1, 2, 4, 8, …

© MITB, SMU

Pattern Recognition

- Finding general terms is a simple pattern recognition process.
- Machine learning techniques are designed to recognize more complex patterns, e.g., how to recognize bicycles in a picture.



- Machine learning mainly deals with implicit patterns
- In this course, our focus is on explicit patterns
  - patterns that can be expressed in natural languages

© MITB, SMU

Pattern recognition in computational thinking

- The task of pattern recognition in computational thinking:
  - What is the next pattern, given several existing patterns?

Steps:

1. Use a natural language to describe the existing patterns
2. Predict and validate the next pattern
3. Formulate the next pattern in a programming language

- Example: *, ***, *****, *******, …
  - Given: *, ***, *****, what should be the next?

© MITB, SMU

Features that may be in patterns

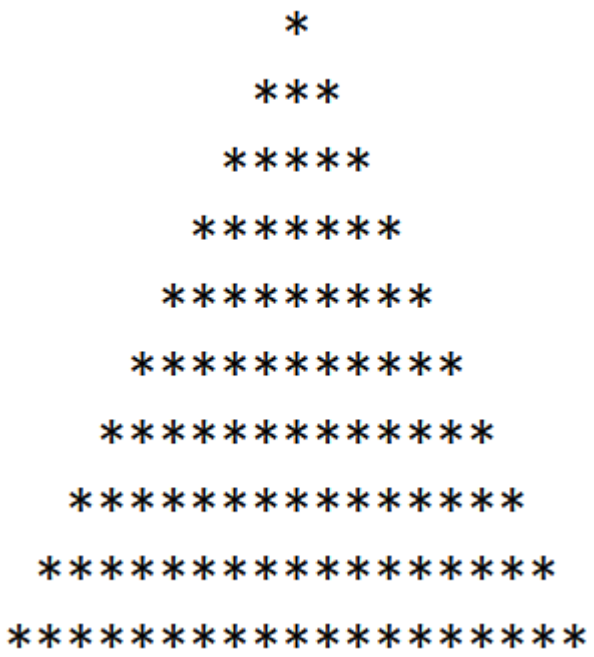
1. Cyclic patterns:
   - ↑, ←, ↓, →, ↑, ←, ↓, →, …
2. The next pattern from the current patterns
   - 1, 2, 4, 8, …
   - The value of the next term is twice the value of the current term.
3. Patterns at the ith position
   - The value at the ith position is 2 raised to the power of i
4. Terminating patterns
   - This pattern could terminate at 1,000,000.

- Very often, a pattern could be a mix of the above patterns.

© MITB, SMU

Exercise 3-1: printing simple patterns

- Print the following pyramid of 10 levels:



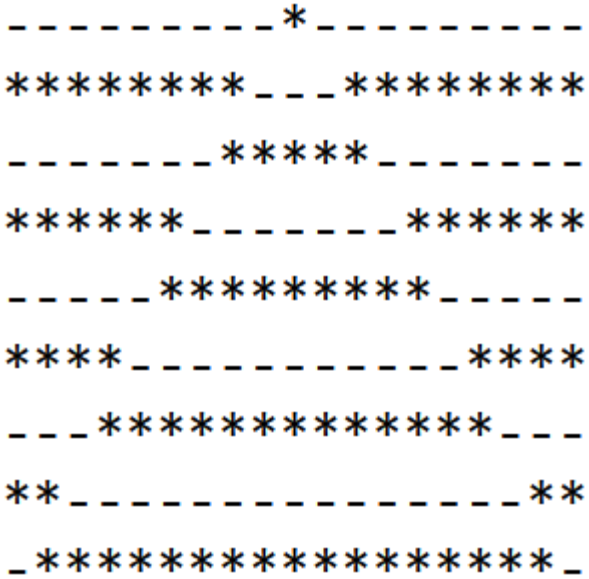
- What are the types of patterns involved?

Curr → Next?	Yes / No	At i^{th} position?	Yes / No
Cyclic patterns?	Yes / No	Terminating patterns?	Yes / No

© MITB, SMU

Exercise 3-2: printing less-simple patterns

- Printing the pattern on the right:



- What are the types of patterns involved?

Curr → Next?	Yes / No	At i^{th} position?	Yes / No
Cyclic patterns?	Yes / No	Terminating patterns?	Yes / No

© MITB, SMU

Recognizing patterns of loops

Curr → Next:

- Out-of-loop variables
- Update out-of-loop variables in each iteration



Either-or

At i^{th} position

- Construct on the fly

```
str1, str2 = ' ' * 9, '*'
for i in range(10):
    print(str1 + str2)
    str1 = str1[:-1]
    str2 += '*' * 2
```

```
for i in range(10):
    print(' ' * (9-i) + '*' * (2*i+1))
```


© MITB, SMU

Recognizing patterns of loops

Cyclic patterns

- Optional
- The pattern of cyclic patterns?

```
str1, str2 = '-' * 9, '*'
for i in range(9):
    print(str1 + str2 + str1)
    if i % 2 == 0:
        str1 = '*' * (len(str1) - 1)
        str2 = '-' * (len(str2) + 2)
    else:
        str1 = '-' * (len(str1) - 1)
        str2 = '*' * (len(str2) + 2)
```

Terminating patterns

- Compulsory
- How is it implemented?
  - A `for` loop iterates each element in the container until it reaches the end
  - This means the loop exits normally

© MITB, SMU

Wooclap!

To print the pattern, what should be filled in to the blank?

```
i, j = 1, 9
while i <= 9:
    if _____________:
        print('*'*i + '-'*j)
    else:
        print('-'*i + '*'*j)
    i = i + 1
    j = j - 1
```

```
*---------
--********
***-------
----******
*****-----
------****
*******---
--------**
*********-
```

While Loops

while loops

- Repeat the loop until the while condition fails
- Basic structure:

```
while test-condition on variable x:
    do something in the loop
    update variable x
```

- If `variable x` is not updated, the while loop may run forever
  - In general, DO NOT design infinite loops
  - Likely a bug when a loop does not end

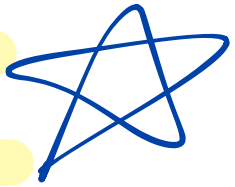
```
>>> summ, i = 0, 0
>>> while i <= 100:
...     summ += i
...     i += 1
>>> print(summ)
5050
>>> ls = ['i', 'like', 'python']
>>> while ls:
...     print(ls.pop(0))
i
like
python
```


© MITB, SMU

Null value for conditionals

- None, False, 0, empty string and empty containers are evaluated as `False`

- `NaN` is evaluated as `True`, but `NaN == NaN` is evaluated as `False`



```
>>> while 0:
...     print('this is evaluated as False')
>>> while '':
...     print('this is evaluated as False')
>>> while []:
...     print('this is evaluated as False')
>>> while {}:
...     print('this is evaluated as False')
>>> while float('NaN') == float('NaN'):
...     print('this is evaluated as False')
```


© MITB, SMU

break and continue

- `break` : exit from the inner most loop, may still stay in the outer loop
- `continue` : go to the while condition of the inner most loop
- Basic structure:

```
while test-condition on variable x:
    do something in the loop
    if condition A:
        continue
    if condition B:
        break
    update variable x
```


© MITB, SMU

Difference between break and continue


```
>>> i = 0
>>> while i < 8:
...     i += 1
...     print('value of i is now', i)
...     if i == 5:
...         break
...     i += 1
value of i is now 1
value of i is now 3
value of i is now 5
```

break → exit loop.

update variable

```
>>> i = 0
>>> while i < 8:
...     i += 1
...     print('value of i is now', i)
...     if i == 5:
...         continue
...     i += 1
value of i is now 1
value of i is now 3
value of i is now 5
value of i is now 6
value of i is now 8
```


© MITB, SMU

Wooclap!

What is the output of the below code?

```
i = 0
while i < 10:
    i += 1
    if i % 2 == 0:
        continue
    if i % 4 == 0:
        break
print(i)
```

goes back to here

} here never runs.

a. 0

b. 2

c. 4

d. 10

Exercise 3-3: Relative luminance

- Relative luminance is a measure of the brightness of a color, considering the varying sensitivity of the human eye to different wavelengths of light.
- The values are normalized as 0.0 to 1.0 (or 1 to 100), with 1.0 (or 100) representing white, calculated with this formula:

$$\text{Rel Lumin} = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$$

- Select the brightest color (highest relative luminance value) from a list of RGB color code.

en.wikipedia.org/wiki/Relative_luminance

Color	HTML / CSS Name	Hex Code #RRGGBB	Decimal Code (R,G,B)
	Black	#000000	(0,0,0)
	White	#FFFFFF	(255,255,255)
	Red	#FF0000	(255,0,0)
	Lime	#00FF00	(0,255,0)
	Blue	#0000FF	(0,0,255)
	Yellow	#FFFF00	(255,255,0)
	Cyan / Aqua	#00FFFF	(0,255,255)
	Magenta / Fuchsia	#FF00FF	(255,0,255)
	Silver	#C0C0C0	(192,192,192)
	Gray	#808080	(128,128,128)
	Maroon	#800000	(128,0,0)
	Green	#008000	(0,128,0)
	Navy	#000080	(0,0,128)
	Olive	#808000	(128,128,0)
	Teal	#008080	(0,128,128)
	Purple	#800080	(128,0,128)

© MITB, SMU

Result from the patterns

- Do not forget about the computational task
  - How does each iteration interact with the resultant variable outside the loop?
- Typical variables: `curr_max`, `found`

```
>>> curr_max, max_idx = float('-inf'), -1
>>> for i in range(len(ls)):
...     if ls[i] > curr_max:
...         curr_max, max_idx = ls[i], i
>>> print('max value', curr_max, 'at index', max_idx)
```

```
>>> found, key, idx = False, 628, -1
>>> for i in range(len(ls)):
...     if ls[i] == key:
...         found, idx = True, i
>>> if found:
...     print('found at index', idx)
... else:
...     print('key not found in list')
```


© MITB, SMU

Exercise 3-4: Checking palindrome

- Task: check if a string is a palindrome
- Implement it with `while` loop

- Palindrome: Strings that read the same forward and backward.

- What are the types of patterns involved?

Curr → Next?	Yes / No	At i^{th} position?	Yes / No
Cyclic patterns?	Yes / No	Terminating patterns?	Yes / No

© MITB, SMU

Exercise 3-5: Maximal palindromes

- List all maximal palindromes

- Maximal: "largest, or longest", cannot grow to a superset ("something larger") with the similar structure

see extra slides.

Curr → Next?	Yes / No	At i^{th} position?	Yes / No
Cyclic patterns?	Yes / No	Terminating patterns?	Yes / No

© MITB, SMU

Wooclap!

What is the output of the below code?

```
for i in range(4):
    for j in range(4):
        if i < j:
            print(i, j, end='; ')
```

a. 0 1; 0 2; 0 3; 1 1; 1 2; 1 3; 2 1; 2 2; 2 3;

b. 2 3; 2 2; 2 1; 1 3; 1 2; 1 1; 0 3; 0 2; 0 1;

c. 0 1; 0 2; 0 3; 1 2; 1 3; 2 3;

d. 1 0; 2 0; 3 0; 2 1; 3 1; 3 2;

Exercise 3-6: Circle packing problems

- Social distancing: In vaccination centers, people are required to sit down at a hall for half an hour after taking the vaccine. The social distance of 1 meter should be observed to ensure safety measures. However, we would like to place as many chairs as possible in the hall to handle largest possible crowd.
- Given the shape of a hall, your task is to compute the maximum number of chairs to be placed with pairwise distance at least 1 meter apart.
- To simplify the problem, you may ignore the size of the chairs, and consider them as points which are at least 1 meter apart.

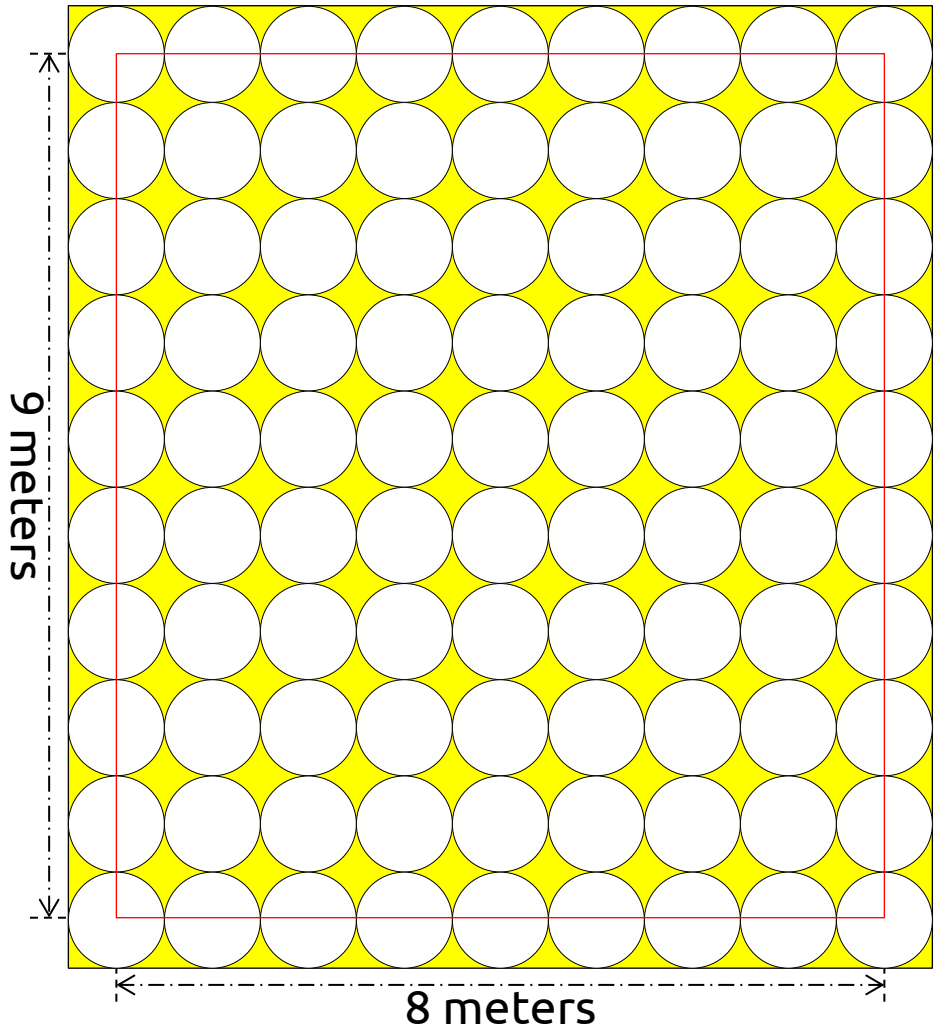
How is the above social distancing problem related to a circle packing problem?

© MITB, SMU

Exercise 3-6: Circle packing in rectangles

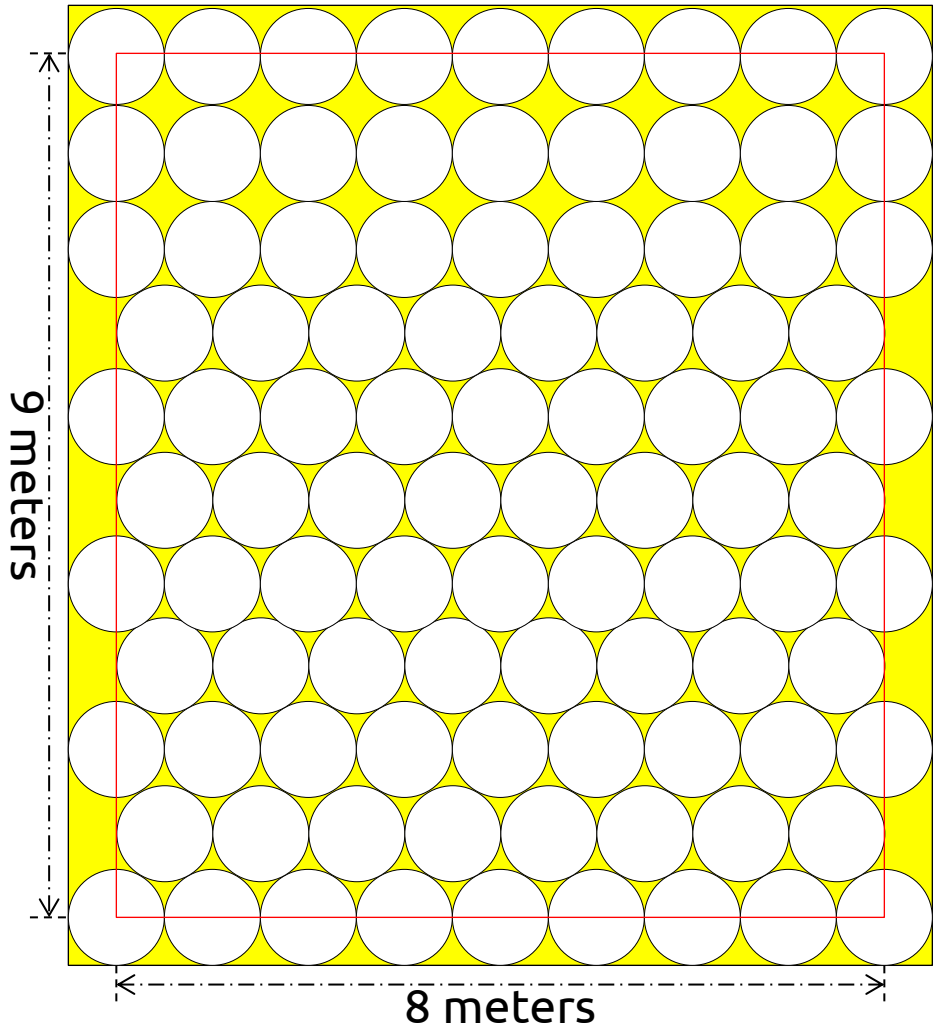
- It is a difficult question to pack circles in an arbitrary shape, e.g., circles, but relatively easier to pack circles of the same radius into rectangles.
- Each chair is modeled as a circle of radius 0.5 meters.
- If the rectangle hall is enlarged by 0.5 meters on all four sides,

the maximum number of chairs you can place	=	the number of circles you can pack in the enlarged rectangle.

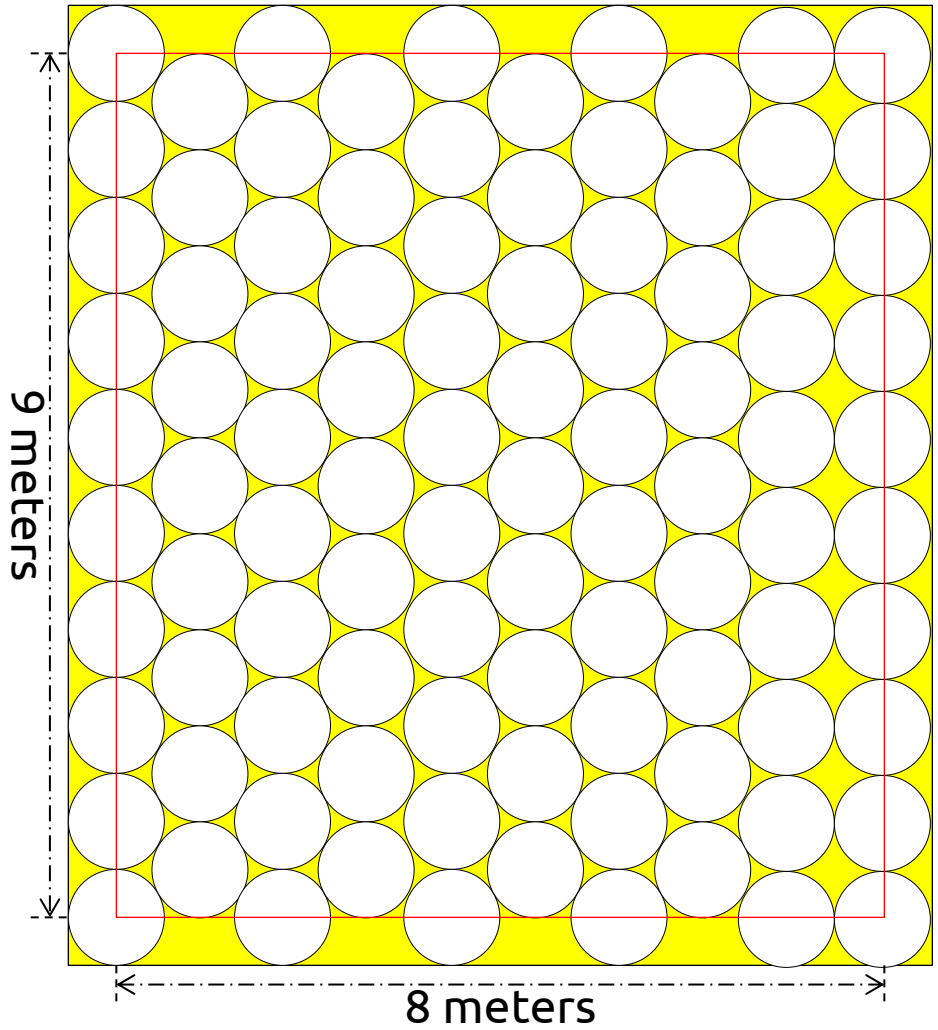


© MITB, SMU

Exercise 3-6: packing more circles



(a) 95 circles



(b) 96 circles

https://medium.com/mitb-for-all/optimal-packing-problem-code-with-explanation-5cc775f73850

© MITB, SMU

Exercise 3-7: End-of-month recurring transfers

- Task: list the dates of transfers between two given month inclusive, assuming recurring transfers are made monthly at the end of each month.

- Irregular patterns: the last day of each month is different.

- Use "Curr → Next?" or "At i^{th} position" ?

© MITB, SMU

Exercise 3-8: Finding primes

- Task: list all primes under 10000
- Definition of primes: an integer with exactly two distinct divisors: 1 and itself.
- Build the naïve solution with double loops, and then consider improving it

© MITB, SMU

Exercise 3-8: Terminating patterns

- Terminating patterns can be optimized
- Observation 1: As long as one non-trivial divisor (trivial = 1 or itself) is found, the number is confirmed to be a non-prime (or a composite)
- Observation 2: Even integers can be skipped, only need to test if an odd integer is divisible by another odd integer
  - Even integers greater than 2 are all composite numbers
  - Even integers cannot divide odd integers

© MITB, SMU

Exercise 3-8: Terminating patterns

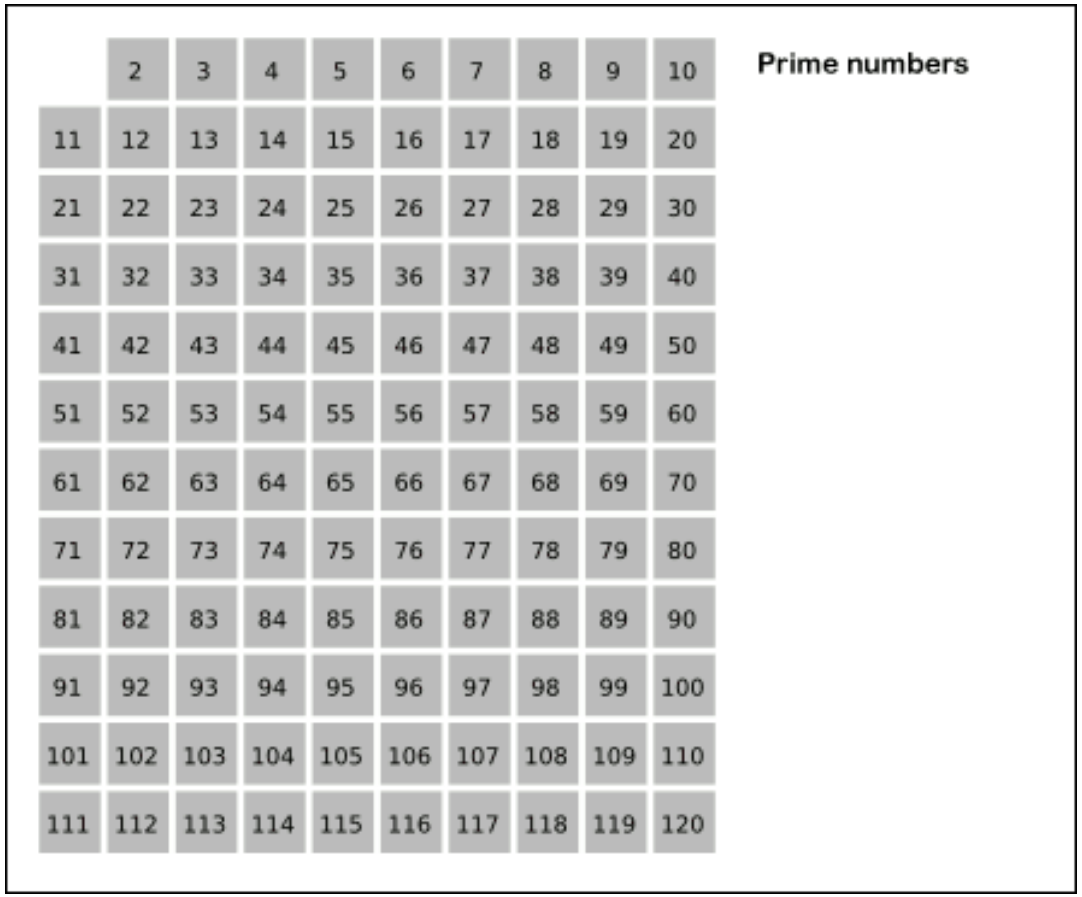
- Observation 3: Non-prime (composite) odd numbers, e.g., 9, 15 or 21, can be skipped
  - By the time we try using 15 to divide an integer, we must have tried both 3 and 5, and both failed, therefore, it is definite that 15 does not divide that integer.

- Observation 4: Divisors typically come in pairs, except for perfect squares, where the square root is a divisor that pairs with itself.
  The smallest non-trivial divisor should not exceed the square root.

© MITB, SMU

Exercise 3-9: the Sieve of Eratosthenes Optional

- Initially, all integers greater than 1 are possible primes
- Starting from 2, cross out all multiple of this number other than itself
  - First, 4, 6, 8, 10, … are crossed out
  - 6, 9, 12, 15, … are crossed out when processing 3
- If an integer is not crossed out, it is guaranteed to be a prime number.



en.wikipedia.org/wiki/Sieve_of_Eratosthenes

© MITB, SMU

Summary

- Pattern recognition: a crucial aspect of computational thinking
  - Programming often involves recognizing patterns that can be described
  - More advanced techniques, e.g., machine learning, are designed for complicated patterns

- When recognizing patterns, focus on either "Curr → Next" or find a rule to describe the pattern at i^{th} position
  - This determines the construction of loops and the handling of iterations

- Cyclic patterns require `if-elif-else` conditional switch in the iterations

- Terminating patterns matter with the number of iterations

© MITB, SMU

SMU
SINGAPORE MANAGEMENT UNIVERSITY

School of
Computing and
Information Systems

Questions?

Thank You

